Reporte Técnico de Actividades Práctico-Experimentales Nro. 05

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Alejandro Padilla
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	05
Tipo	Individual
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	Ing. José O. Guamán Q.
Fecha	2026-10-30
Horario	Jueves 10h30 a 11h30; Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.
Los objetivos específicos son:

1. Implementar un generador LCG en Python.
2. Implementar el mismo algoritmo en Java.
3. Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador con Linux.
- Python 3.14, con Matplotlib para las gráficas y pytest para las pruebas.
- Java/JDK 21.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

El generador congruencial lineal obtiene cada valor a partir del anterior:

$$X_{n+1} = (a \cdot X_n + c) \text{ mód } m$$

$$U_n = \frac{X_n}{m}$$

X(0) es la semilla. U(n) cae en [0, 1) y se toma como una muestra de una uniforme entre 0 y 1. La calidad del generador depende de los parámetros y de la semilla seleccionada: los parámetros fijan el período y la semilla, el punto de partida.

4.2. Parámetros

Se usaron los valores de la guía: a = 1664525, c = 1013904223, $m = 2^{32}$ y semilla = 12345.

4.3. Implementación en Python

Se generaron 10 000 valores en Python. El código está organizado en modelo, vista y controlador (paquete simulacion_ape). El generador es la función lcg_states del modelo; devuelve X(1), X(2), ... sin incluir la semilla:

```python
def lcg_states(config: LcgConfig) -> list[int]:
    states = []
    x = config.seed
    for _ in range(config.n):
        x = (config.a * x + config.c) % config.m
        states.append(x)
    return states
```

Después, U(n) se calcula dividiendo cada estado entre m.

4.4. Implementación en Java

El mismo algoritmo está en la clase Lcg. Usa long porque a · X(n) no cabe en un int; sí cabe en un long, ya que $a < 2^{21}$ y $X(n) < 2^{32}$. Math.floorMod evita residuos negativos:

```
public long next() {
    state = Math.floorMod(a * state + c, m);
    return state;
}
```

El programa de Python compila y ejecuta Lcg.java con los mismos parámetros, lee los valores que imprime y los compara con los suyos.

4.5. Pasos seguidos

1. Se generaron 10 000 valores con los parámetros de la guía, en Python y en Java.
2. Se compararon las primeras 20 observaciones y después los 10 000 valores completos.
3. Se graficó la secuencia.
4. Se analizó el período con el teorema de Hull-Dobell y se midió en módulos pequeños.
5. Se cambió la semilla a 54321 y a 1, y se compararon los resultados entre las tres.

5. Resultados

5.1. Comparación de las primeras 20 observaciones

Python y Java generan los mismos valores. El primero es X(1) = 87628868, es decir U(1) = 0.02040269. La Tabla 1 muestra las 20 primeras observaciones de cada programa; la última columna indica si coinciden.

Tabla 1. Primeras 20 observaciones de Python y de Java

n	X(n) Python	X(n) Java	U(n) Python	U(n) Java	Igual
1	87628868	87628868	0.02040269	0.02040269	sí
2	71072467	71072467	0.01654785	0.01654785	sí
3	2332836374	2332836374	0.54315579	0.54315579	sí
4	2726892157	2726892157	0.63490406	0.63490406	sí
5	3908547000	3908547000	0.91002951	0.91002951	sí
6	483019191	483019191	0.11246167	0.11246167	sí
7	2129828778	2129828778	0.49588941	0.49588941	sí
8	2355140353	2355140353	0.54834884	0.54834884	sí
9	2560230508	2560230508	0.59610012	0.59610012	sí
10	3364893915	3364893915	0.78345042	0.78345042	sí
11	171172990	171172990	0.03985432	0.03985432	sí
12	3194601925	3194601925	0.74380122	0.74380122	sí

n	X(n) Python	X(n) Java	U(n) Python	U(n) Java	Igual
13	4148119648	4148119648	0.96580937	0.96580937	sí
14	316399679	316399679	0.07366754	0.07366754	sí
15	3004788882	3004788882	0.69960693	0.69960693	sí
16	1976948425	1976948425	0.46029417	0.46029417	sí
17	1702883732	1702883732	0.39648352	0.39648352	sí
18	4121112547	4121112547	0.95952129	0.95952129	sí
19	1744294886	1744294886	0.40612530	0.40612530	sí
20	4092090893	4092090893	0.95276416	0.95276416	sí

El programa comparó además los 10 000 valores completos: 0 diferencias en los 10 000 valores.

5.2. Gráfica de la secuencia

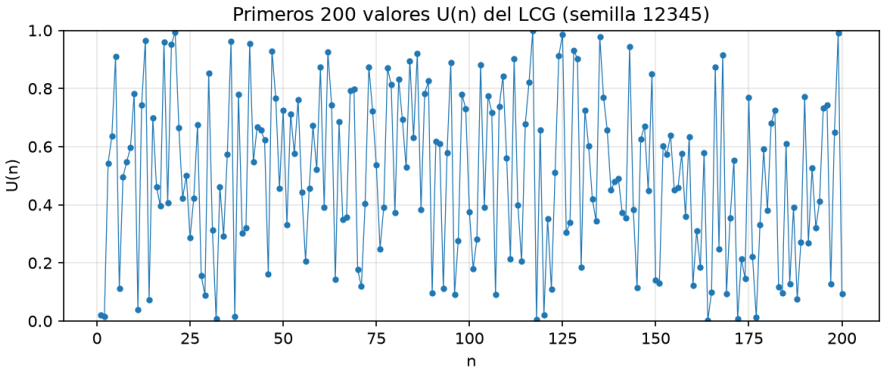


Figura 1: Primeros 200 valores U(n) del LCG con semilla 12345

Los valores saltan por todo el rango entre 0 y 1, sin tendencia ni patrón que se note a simple vista.

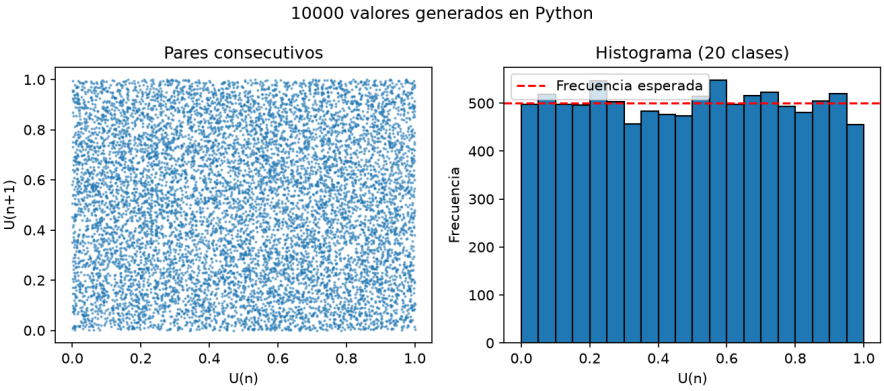


Figura 2: Pares consecutivos (U(n), U(n+1)) e histograma de 20 clases para los 10 000 valores

Los pares llenan el cuadrado sin alinearse en rectas, y las 20 barras del histograma quedan cerca de la frecuencia esperada, 500 cada una.

5.3. Análisis del período

Con los parámetros de la guía se verificaron las tres condiciones del teorema de Hull-Dobell:

- c = 1013904223 y $m = 2^{32}$ son primos relativos: se cumple.
- a − 1 = 1664524 es divisible por el único factor primo de m, que es 2: se cumple.
- Como 4 divide a m, también 4 divide a a − 1: se cumple.

Por eso el período es completo: $\tau = m = 4294967296$. En la muestra de 10 000 valores ningún valor se repite, lo que es coherente con ese período.

Para ver el efecto de los parámetros se midió el período con un módulo pequeño:

- a = 5, c = 3, m = 16, semilla 7: Hull-Dobell se cumple y el período es 16, de 16 posibles.
- a = 4, c = 2, m = 16, semilla 7: Hull-Dobell no se cumple. La secuencia pasa por 14 y queda fija en 10, así que el ciclo al que llega tiene largo 1, de 16 posibles.

5.4. Cambio de semilla

Se repitió la generación con las semillas 54321 y 1, manteniendo a, c y m. La Tabla 2 compara las tres con los valores ideales de una uniforme: media 0.5 y varianza $1/12 = 0.0833$.

Tabla 2. Estadísticos de 10 000 valores según la semilla

semilla	media	varianza	correlación entre U(n) y U(n+1)
12345	0.4988	0.0830	-0.0153
54321	0.4989	0.0836	-0.0020
1	0.4974	0.0834	+0.0044

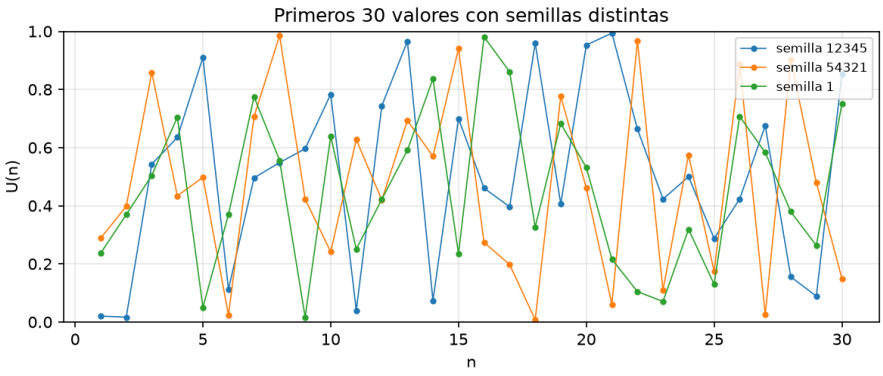


Figura 3: Primeros 30 valores U(n) con las semillas 12345, 54321 y 1

Cada semilla produce una secuencia distinta desde el primer valor (0.0204, 0.2883 y 0.2365), pero las tres tienen media y varianza cercanas a las ideales y una correlación cercana a cero.

6. Preguntas de Control

1. ¿Qué significa pseudoaleatorio?
Que los números salen de un algoritmo determinista: con la misma semilla se obtiene siempre la misma secuencia. Aun así parecen aleatorios: un LCG con buenos parámetros puede pasar pruebas estadísticas de uniformidad e independencia, y además se puede reproducir, que es lo que pide una simulación.

2. ¿Qué función cumple la semilla?
Es el valor inicial X(0) y determina por completo la secuencia. Con la semilla 12345, Python y Java dan los mismos valores; con 54321 y 1 salen otras secuencias, con estadísticos parecidos.

3. ¿Qué significa período?
Es la longitud del ciclo en el que termina la secuencia: cuántos valores distintos genera antes de volver a uno ya visto dentro de ese ciclo. Nunca supera a m. Con los parámetros de la guía, $\tau = 2^{32} = 4294967296$.

4. ¿Qué ocurre si se utilizan parámetros inadecuados?
El período se acorta. Con m = 16, los parámetros a = 4, c = 2 dieron período 1 en lugar de 16. Según las diapositivas de la semana 5, también pueden aparecer correlaciones entre valores, que se ven como puntos alineados en la gráfica de pares consecutivos.

5. ¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?
Permite comprobar que la implementación es correcta: si los 10 000 valores coinciden, es poco probable que ambas tengan el mismo error. Además obliga a cuidar detalles de cada lenguaje, como el desborde en Java descrito en su implementación.

7. Conclusiones

- Las implementaciones en Python y en Java generan la misma secuencia: las 20 primeras observaciones de la Tabla 1 coinciden y los 10 000 valores no tienen diferencias.
- Los parámetros de la guía cumplen las tres condiciones de Hull-Dobell, así que el generador recorre sus 2^{32} estados antes de repetirse. En el módulo pequeño, solo los parámetros que las cumplen alcanzaron el período completo.
- Con las tres semillas, la media, la varianza y la correlación entre valores consecutivos quedaron cerca de lo esperado para una uniforme. Cambiar la semilla cambia la secuencia, pero no su calidad.
- Estas medidas no demuestran uniformidad ni independencia; para eso hacen falta pruebas estadísticas formales, que esta práctica no incluye.

8. Recomendaciones

- Verificar las condiciones de Hull-Dobell antes de usar parámetros nuevos.
- Usar este generador solo para simulación; no es adecuado para criptografía.

Bibliografía / Referencias

[1] J. O. Guamán, *Generadores pseudoaleatorios: el Generador Congruencial Lineal (LCG), de la fórmula al fallo*, Diapositivas de clase, semana 5, asignatura Simulación, Carrera de Computación, Universidad Nacional de Loja, Loja, Ecuador, 2026.

9. Anexos

El código de Python (paquete simulacion_ape, con sus pruebas en pytest) y el de Java (Lcg.java) están en la carpeta codigo del repositorio de la asignatura.

Declaración de uso de IA Generativa

Se usaron herramientas de IA generativa como asistente durante la práctica. Las tablas siguientes resumen las herramientas y las instrucciones dadas.

Tabla A. Herramientas utilizadas

Herramienta	Uso
el Gentleman (entorno Pi), con un modelo Claude de Anthropic	Leer la guía y las diapositivas, escribir el código, generar las gráficas y redactar y maquetar el informe
Python 3.14 (Matplotlib, pytest)	Implementación del LCG, gráficas y pruebas
Java/JDK 21	Implementación del mismo algoritmo y comparación
LaTeX	Maquetación del documento

Tabla B. Prompts por iteración (resúmenes de las instrucciones, no texto literal)

Iteración	Prompt (resumen)
1. Lectura de la guía y de las diapositivas	Lee la guía de la APE 5 y las diapositivas de la semana 5 y resume qué pide la práctica y qué resultados se esperan.
2. Pruebas del generador	Calcula a mano el primer valor del LCG con los parámetros de la guía y comprueba la semilla y el período antes de programar.
3. Implementación	Implementa el LCG en Python con estructura modelo, vista y controlador y el mismo algoritmo en Java, cuidando el desborde en Java.
4. Comparación y período	Compara las secuencias de Python y Java (primeras 20 observaciones y los 10 000 valores) y analiza el período con el teorema de Hull-Dobell y con módulos pequeños.

Iteración	Prompt (resumen)
5. Gráficas y semillas	Grafica la secuencia y repite la generación con las semillas 54321 y 1, comparando media, varianza y correlación.

Todo el código, los resultados y el texto fueron revisados, ejecutados y validados por el autor, quien asume la responsabilidad sobre la calidad y veracidad del trabajo.

Alejandro Padilla
Autor